Reverse Engineering Smart Charge

Huawei MateBook D 15 (BohrD-WDH9D) – Analisi ACPI / Embedded Controller e implementazione Linux

Versione estesa • Data: 2026-03-07

Autore: Guido Sorarù (sorguido@gmail.com)

Indice generale

Estratto....1
1. Sistema e contesto....1
2. Obiettivo e definizione del problema....2
3. Metodologia di reverse engineering....2
3.1 Estrazione tabelle ACPI....2
3.2 Ricerca dei metodi e correlazione con EC....2
3.3 Validazione sperimentale....2
4. Metodi ACPI rilevanti....2
5. Registri EC e semantica osservata....3
5.1 Diagrammi....3
6. Risultati sperimentali principali....4
6.1 Sequenza funzionante (40/70)....4
6.2 Lettura registri EC e sysfs....4
7. Persistenza: reboot vs shutdown....4
8. Tooling: Smart_Charge.py....5
Appendice A — Comandi ACPI testati (estratto completo)....5
Appendice B — Timeline della ricerca....6
9. Conclusioni e lavoro futuro....7

Estratto

Questa relazione documenta un percorso completo di reverse engineering del meccanismo di limitazione della carica batteria (“Smart Charge”/Battery Protection) implementato nel firmware dei notebook Huawei MateBook.
L’obiettivo pratico è replicare su Linux la funzione disponibile su Windows tramite Huawei PC Manager, configurando soglie di carica (es. stop al 70%) e comprendendo i limiti reali della piattaforma.

L’analisi include estrazione e decompilazione delle tabelle ACPI (DSDT/SSDT), identificazione dei metodi AML rilevanti, correlazione con i registri dell’Embedded Controller (EC), validazione sperimentale tramite acpi_call e osservazione del comportamento di persistenza (reboot vs shutdown).

1. Sistema e contesto

Hardware: Huawei MateBook D 15 (BohrD-WDH9D), anno 2021.
Sistema operativo: Linux Mint. Kernel: serie 6.x (test su 6.17).
Batteria ACPI: BAT1. EC (Embedded Controller) osservato in ACPI al path: _SB.PC00.LPCB.EC0.

2. Obiettivo e definizione del problema

Obiettivo funzionale:
controllare il limite superiore di carica (es. stop al 70%) in modo analogo a Huawei PC Manager su Windows.

Vincoli: assenza di Windows (impossibile sniffing diretto del traffico WMI), necessità di operare via ACPI/EC su Linux.

Problema iniziale:
il driver huawei_wmi espone un nodo sysfs per soglie di carica, ma la scrittura fallisce con errore I/O, consentendo solo lettura dello stato.

3. Metodologia di reverse engineering

3.1 Estrazione tabelle ACPI

Sono state estratte le tabelle ACPI con acpidump e poi separate con acpixtract.
Successivamente è stata eseguita la decompilazione AML→DSL con iasl. Durante la decompilazione sono emersi warning/AE_NOT_FOUND legati a riferimenti esterni (tipici in dump parziali), ma i file DSL risultanti sono utilizzabili per analisi statica.

3.2 Ricerca dei metodi e correlazione con EC

La ricerca è stata condotta individuando metodi con nomi suggestivi (SBTT/GBTT/SBCM/GBCM) e verificando all'interno del DSL le scritture ai registri EC.
La correlazione è stata confermata sperimentalmente leggendo direttamente i registri EC via acpi_call prima e dopo l'invocazione dei metodi.

3.3 Validazione sperimentale

Per l'esecuzione runtime dei metodi ACPI è stato installato acpi_call-dkms.
I test consistono in:
- invocare metodi con buffer specifici,
- leggere i ritorni su /proc/acpi/call,
- osservare i registri EC (CHAM/BCCL/BCCH…),
- verificare l'effetto su
/sys/class/power_supply/BAT1/* (status/capacity/current_now).

4. Metodi ACPI rilevanti

Metodi principali individuati nel DSDT e/o richiamati via WMI table:
- \SBTT: imposta le soglie batteria (scrive BCCL/BCCH)
- \GBTT: legge le soglie
- \SBCM: imposta la modalità di carica (scrive CHAM e può impostare soglie)
- \GBCM: legge la modalità/parametri correnti
- \SBAC / \GBAC: controlli AC correlati (osservati ma non determinanti per l'obiettivo)
- \SODP / \SPLV / \SSAP / \SSUP / \SBPP: metodi testati come candidati per sblocco/override (nessun effetto osservato sui limiti).

5. Registri EC e semantica osservata

Dalle evidenze statiche e sperimentali emergono i seguenti registri chiave:
CHAM (8-bit): selettore modalità carica (“charge mode”).
BCCL (8-bit): soglia inferiore (valore spesso 0x28=40).
BCCH (8-bit): soglia superiore (valore tipico 0x46=70).
BADS (bit): flag AC disable/state.
SPMD (bit): modalità power management associata a specifici rami di logica in DSDT.

Nota importante: nei test reali la soglia inferiore non risulta determinante per l’avvio della carica. Se la batteria è al 50% e si collega l’alimentatore, la carica parte immediatamente e arriva fino alla soglia superiore.

5.1 Diagrammi

Figura 1 — Catena di controllo OS → ACPI → EC → firmware → charger → batteria.

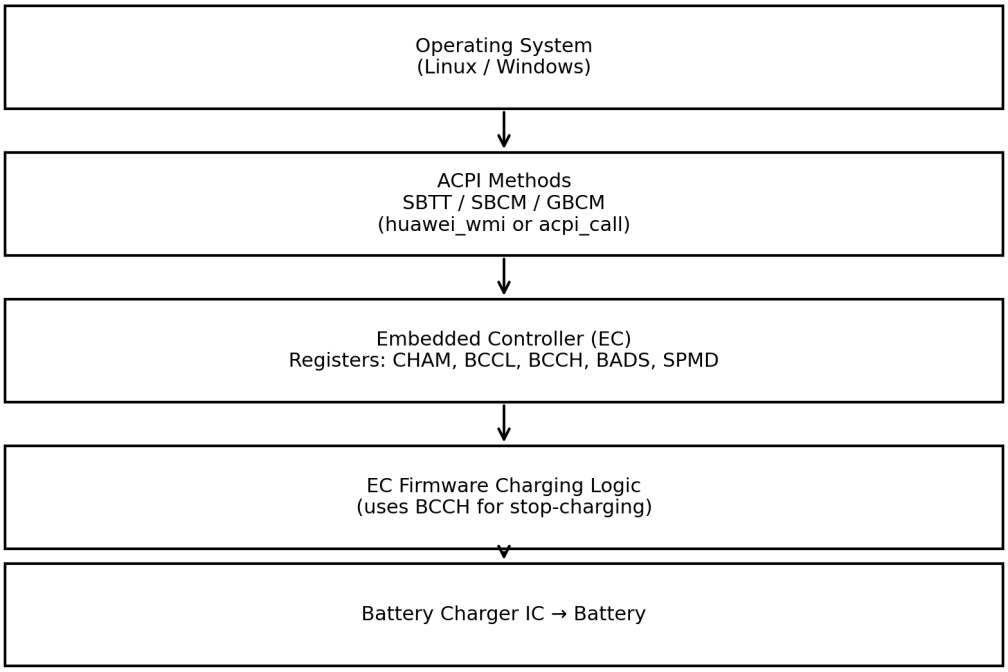


Figura 2 — Macchina a stati semplificata della carica.

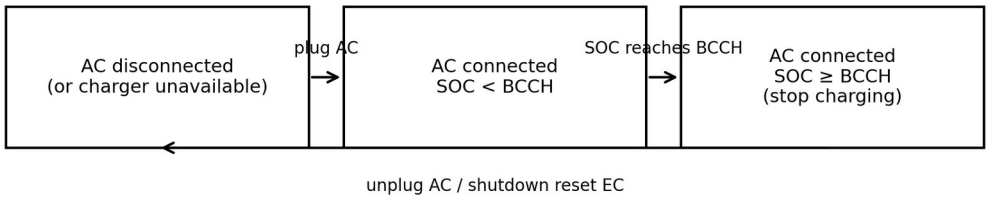


Figura 3 — Mappa registri EC rilevanti.

Register	Meaning	Observed / Typical values
CHAM	Charge mode / policy selector	0x04 (Smart Charge active); 0x00 (limits cleared)
BCCL	Lower threshold parameter	0x28 (40%) — stored, but does not gate charging
BCCH	Upper threshold parameter	0x46 (70%) — used to stop charging
BADS	AC disable / battery AC state flag	0x00 in observed snapshots
SPMD	Power management mode flag	0x00 in observed snapshots

6. Risultati sperimentali principali

6.1 Sequenza funzionante (40/70)

È stata identificata una sequenza stabile che imposta con successo la modalità “Smart Charge” con stop a 70%.
La sequenza usa:
1) \SBTT per scrivere le soglie
2) \SBCM per impostare CHAM e confermare parametri
3) \GBCM per leggere lo stato risultante

```
sudo modprobe acpi_call
echo '\SBTT (Buffer(){0x00,0x00,0x28,0x46})' | sudo tee /proc/acpi/call
>/dev/null
echo '\SBCM (Buffer(){0x00,0x00,0x01,0x00,0x28,0x46})' | sudo tee
/proc/acpi/call
sudo cat /proc/acpi/call
echo '\GBCM (Buffer(){0x00})' | sudo tee /proc/acpi/call >/dev/null
sudo cat /proc/acpi/call
```

6.2 Lettura registri EC e sysfs

I registri EC risultano coerenti con la modalità attiva:
CHAM=0x04, BCCL=0x28, BCCH=0x46.
A livello di sysfs, una volta raggiunto il 70%:
BAT1/status → Not charging BAT1/current_now → 0

7. Persistenza: reboot vs shutdown

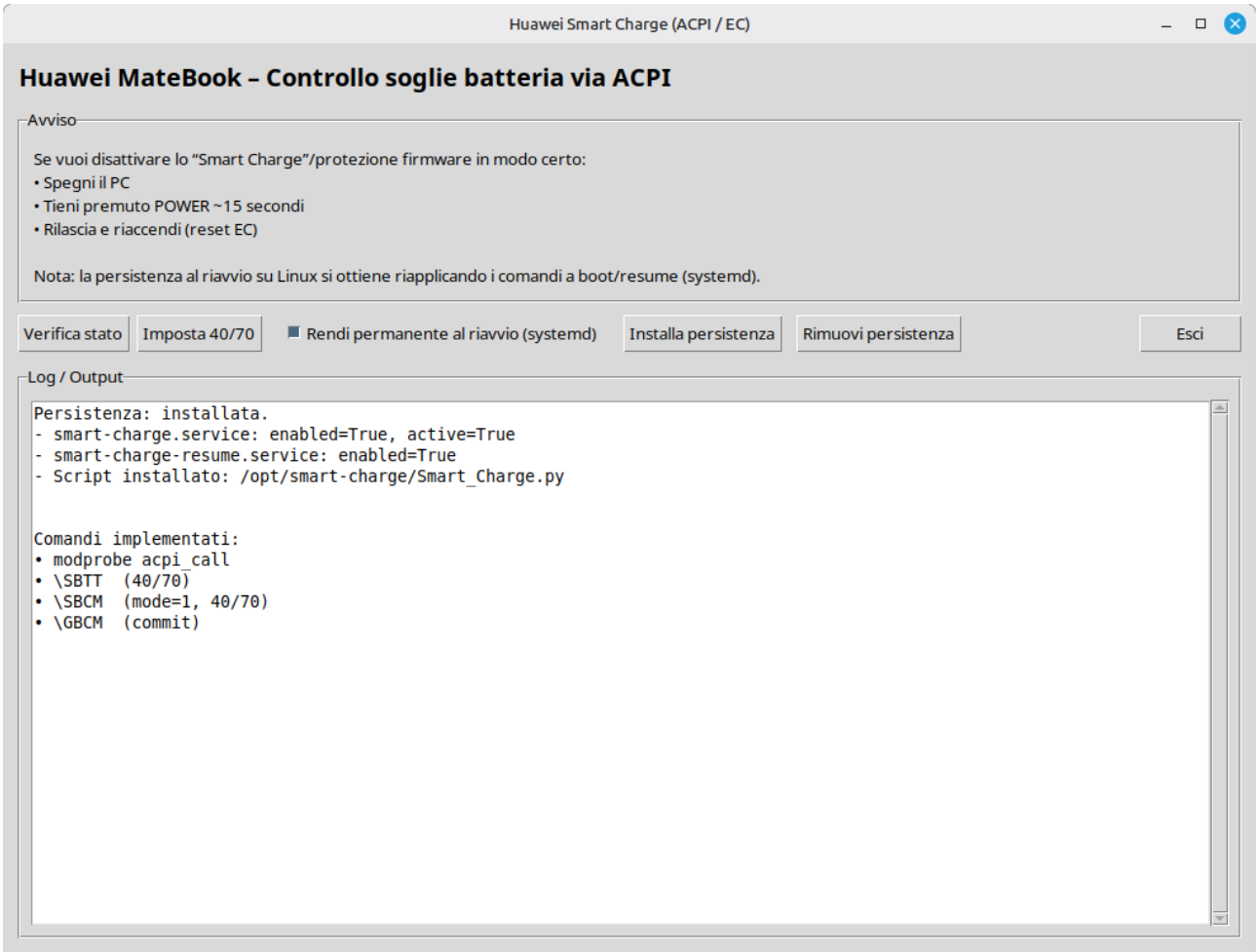
Risultato cruciale: le impostazioni “Smart Charge” persistono dopo reboot, ma non dopo shutdown completo.

Interpretazione tecnica:
la configurazione sembra risiedere nello stato volatile dell'EC o in un contesto runtime non salvato su storage persistente. Implicazione: per avere il limite attivo "sempre", è sufficiente applicarlo ad ogni boot (systemd o autostart).

8. Tooling: Smart_Charge.py

È stato realizzato uno script Python con GUI (Smart_Charge.py) per automatizzare:
- caricamento modulo acpi_call
- esecuzione della sequenza funzionante
- lettura di ritorni ACPI e parametri di batteria
- messaggio informativo: per disattivare completamente Smart Charge talvolta potrebbe essere necessario spegnere e tenere premuto Power ~15s (reset EC).

Nota: la GUI richiede una sessione grafica (variabile DISPLAY valorizzata).
In esecuzione headless comparirà l'errore TclError "no display name".



Appendice A — Comandi ACPI testati (estratto completo)

Di seguito una raccolta dei comandi ACPI effettivamente utilizzati durante i test e citati nella ricerca.
L'elenco è utile per riprodurre l'esperimento e per ulteriori investigazioni su firmware/EC.

```
# Load module
sudo modprobe acpi_call

# Working configuration (40/70)
echo '\SBTT (Buffer(){0x00,0x00,0x28,0x46})' | sudo tee /proc/acpi/call
>/dev/null
echo '\SBCM (Buffer(){0x00,0x00,0x01,0x00,0x28,0x46})' | sudo tee
/proc/acpi/call
sudo cat /proc/acpi/call
echo '\GBCM (Buffer(){0x00})' | sudo tee /proc/acpi/call >/dev/null
sudo cat /proc/acpi/call
echo '\GBTT (Buffer(){0x00,0x00,0x00,0x00})' | sudo tee /proc/acpi/call
>/dev/null
sudo cat /proc/acpi/call

# Attempts to set 0/100 (did not persist as expected in Smart Charge
mode)
echo '\SBTT (Buffer(){0x00,0x00,0x00,0x64})' | sudo tee /proc/acpi/call
>/dev/null
echo '\SBCM (Buffer(){0x00,0x00,0x01,0x00,0x00,0x64})' | sudo tee
/proc/acpi/call
sudo cat /proc/acpi/call

# Read EC registers directly
echo '\_SB.PC00.LPCB.EC0.CHAM' | sudo tee /proc/acpi/call >/dev/null;
sudo cat /proc/acpi/call
echo '\_SB.PC00.LPCB.EC0.BCCL' | sudo tee /proc/acpi/call >/dev/null;
sudo cat /proc/acpi/call
echo '\_SB.PC00.LPCB.EC0.BCCH' | sudo tee /proc/acpi/call >/dev/null;
sudo cat /proc/acpi/call
echo '\_SB.PC00.LPCB.EC0.BADS' | sudo tee /proc/acpi/call >/dev/null;
sudo cat /proc/acpi/call
echo '\_SB.PC00.LPCB.EC0.SPMD' | sudo tee /proc/acpi/call >/dev/null;
sudo cat /proc/acpi/call

# Other candidates tested (no effect observed)
echo '\SODP (Buffer(){0x00,0x00,0x00,0x00})' | sudo tee /proc/acpi/call
>/dev/null
echo '\SPLV (0x00)' | sudo tee /proc/acpi/call >/dev/null
echo '\SPLV (0x01)' | sudo tee /proc/acpi/call >/dev/null
echo '\SPLV (0x02)' | sudo tee /proc/acpi/call >/dev/null
echo '\SSAP (0x00)' | sudo tee /proc/acpi/call >/dev/null
echo '\SSAP (0x01)' | sudo tee /proc/acpi/call >/dev/null
echo '\SSAP (0x02)' | sudo tee /proc/acpi/call >/dev/null
echo '\SSUP (0x00)' | sudo tee /proc/acpi/call >/dev/null
echo '\SSUP (0x01)' | sudo tee /proc/acpi/call >/dev/null
echo '\SSUP (0x02)' | sudo tee /proc/acpi/call >/dev/null
echo '\SBPP (0x00)' | sudo tee /proc/acpi/call >/dev/null
echo '\SBPP (0x01)' | sudo tee /proc/acpi/call >/dev/null
echo '\SBPP (0x02)' | sudo tee /proc/acpi/call >/dev/null

# AC behavior (read)
echo '\GBAC (Buffer(){0x00})' | sudo tee /proc/acpi/call >/dev/null;
sudo cat /proc/acpi/call
```

Appendice B — Timeline della ricerca

- Fase 1: osservazione driver huawei_wmi e fallimento scrittura soglie (I/O error)
- Fase 2: estrazione ACPI (acpidump/acpixtract) e decompilazione (iasl)
- Fase 3: identificazione metodi SBTT/GBTT/SBCM/GBCM e registri EC

- Fase 4: test runtime con acpi_call e validazione soglie 40/70
- Fase 5: analisi del blocco a 70% e ipotesi “firmware mode”
- Fase 6: discovery persistenza (reboot mantiene, shutdown resetta)
- Fase 7: sviluppo tool Smart_Charge.py e documentazione

9. Conclusioni e lavoro futuro

Conclusione:
è possibile replicare su Linux la limitazione superiore (stop 70%) tramite metodi ACPI che scrivono su registri EC.
La soglia inferiore appare non utilizzata nella logica di innesco carica, almeno nel comportamento osservato su questa piattaforma.
Il meccanismo non è persistente dopo shutdown, quindi l’automazione all’avvio è l’approccio corretto per ottenere un comportamento costante.

Lavoro futuro consigliato:
- integrazione stabile nel driver huawei_wmi (abilitare scrittura soglie)
- formalizzazione del protocollo WMI (Device HWMI/WMAA) se utile a controlli addizionali
- valutazione di eventi EC e notify BAT1 per aggiornamenti di stato.